

LAPORAN TUGAS 6
SISTEM OPERASI



Dosen pengampu :
Triawan Adi Cahyono M.Kom

Disusun Oleh:
Nama : Putri Suci Rahmadhani
Nim : 2410651008
Kelas : A
Tanggal : 14 November 2025

PRODI TEKNIK INFORMATIKA FAKULTAS TEKNIK
UNIVERSITAS MUHAMMADIYAH JEMBER

2025

DAFTAR ISI

1. TUGAS 1 ANALISIS DEADLOCK

- 1.1. Mengapa program mengalami deadlock?
- 1.2. Kondisi deadlock yang terpenuhi
- 1.3. Resource allocation

2. TUGAS 2 MODIFIKASI PROGRAM

- 2.1. Kode Pemrograman
- 2.2. Testing Output
- 2.3. Analisis Hasil
- 2.4. Perbandingan lock ordering vs try lock

3. TUGAS 3 BANKER'S ALGORITHM

- 3.1. Kode Pemrograman
- 3.2. Testing Output
- 3.3. Analisis Hasil

4. TUGAS 4 KASUS NYATA: DEADLOCK TRANSFER UANG

- 4.1. Kode Pemrograman
- 4.2. Testing Output
- 4.3. Analisis Hasil

5. KESIMPULAN

1. TUGAS 1 ANALISIS DEADLOCK

1.1. Mengapa program mengalami deadlock?

Program mengalami deadlock karena:

- Thread 1 mengambil lock 1, lalu menunggu lock 2.
- Thread 2 mengambil lock 2, lalu menunggu lock 1.
- Kedua thread saling menunggu, tidak ada yang melepaskan lock duluan.
- Hasilnya kedua thread berhenti dan program tidak pernah selesai.

1.2. Kondisi deadlock yang terpenuhi

Deadlock terjadi jika 4 kondisi terpenuhi

Kondisi	Penjelasan
Mutual exclusion	Lock1 dan lock2 hanya boleh dipegang satu thread dalam satu waktu.
Hold and wait	T1 memegang lock1 sambil menunggu lock2, T2 memegang lock2 sambil menunggu lock1
No preemption	Lock tidak bisa direbut secara paksa, harus dilepas secara sukarela
Circular wait	T1 butuh L2, T2 butuh L1

1.3. Resource allocation

Menggunakan symbol : T1, T2 = Thread, L1, L2 = Lock

2. TUGAS 2 MODIFIKASI PROGRAM

2.1. Kode Pemrograman

Versi lock_ordering kedua thread selalu mengambil lock1 dulu baru lock2

```

GNU nano 4.0
#include <stdio.h>
#include <pthread.h>
#include <unistd.h>

pthread_mutex_t lock1 = PTHREAD_MUTEX_INITIALIZER;
pthread_mutex_t lock2 = PTHREAD_MUTEX_INITIALIZER;

// Selalu ambil lock dalam urutan yang sama
void take_locks() {
    pthread_mutex_lock(&lock1);
    pthread_mutex_lock(&lock2);
}

void release_locks() {
    pthread_mutex_unlock(&lock2);
    pthread_mutex_unlock(&lock1);
}

void* thread1(void* arg) {
    printf("Thread1: mulai\n");
    take_locks();
    printf("Thread1: bekerja...\n");
    sleep(1);
    release_locks();
    printf("Thread1: selesai\n");
    return NULL;
}

void* thread2(void* arg) {
    printf("Thread2: mulai\n");
    take_locks();
    printf("Thread2: bekerja...\n");
    sleep(1);
    release_locks();
    printf("Thread2: selesai\n");
    return NULL;
}

int main() {
    pthread_t t1, t2;

    int main() {
        pthread_t t1, t2;
        pthread_create(&t1, NULL, thread1, NULL);
        pthread_create(&t2, NULL, thread2, NULL);

        pthread_join(t1, NULL);
        pthread_join(t2, NULL);

        return 0;
    }
}

```

Versi try_lock

```

#include <stdio.h>
#include <pthread.h>
#include <unistd.h>

pthread_mutex_t lock1 = PTHREAD_MUTEX_INITIALIZER;
pthread_mutex_t lock2 = PTHREAD_MUTEX_INITIALIZER;

void* thread1(void* arg) {
    while (1) {
        pthread_mutex_lock(&lock1);

        if (pthread_mutex_trylock(&lock2) == 0) {
            printf("Thread1: mendapat kedua lock\n");
            sleep(1);
            pthread_mutex_unlock(&lock2);
            pthread_mutex_unlock(&lock1);
            break;
        }
        else {
            printf("Thread1: gagal dapat lock2, ulangi\n");
            pthread_mutex_unlock(&lock1);
            usleep(100000);
        }
    }
    return NULL;
}

void* thread2(void* arg) {
    while (1) {
        pthread_mutex_lock(&lock2);

        if (pthread_mutex_trylock(&lock1) == 0) {
            printf("Thread2: mendapat kedua lock\n");
            sleep(1);
            pthread_mutex_unlock(&lock1);
            pthread_mutex_unlock(&lock2);
            break;
        }
        else {
            printf("Thread2: gagal dapat lock1, ulangi\n");
            pthread_mutex_unlock(&lock2);
            usleep(120000);
        }
    }
    return NULL;
}

int main() {
    pthread_t t1, t2;
    pthread_create(&t1, NULL, thread1, NULL);
    pthread_create(&t2, NULL, thread2, NULL);
    pthread_join(t1, NULL);
    pthread_join(t2, NULL);
    return 0;
}

```

2.2. Testing Output

Versi lock_ordering

```

madha_27@DESKTOP-S58N8I3:~$ nano lock_ordering.c
madha_27@DESKTOP-S58N8I3:~$ gcc lock_ordering.c -o lock_ordering -pthread
madha_27@DESKTOP-S58N8I3:~$ ./lock_ordering
Thread1: mulai
Thread2: mulai
Thread1: bekerja...
Thread1: selesai
Thread2: bekerja...
Thread2: selesai
madha_27@DESKTOP-S58N8I3:~$ |

```

Versi try_lock

```

madha_27@DESKTOP-S58N8I3:~$ nano try_lock.c
madha_27@DESKTOP-S58N8I3:~$ gcc try_lock.c -o try_lock -pthread
madha_27@DESKTOP-S58N8I3:~$ ./try_lock
Thread1: mendapat kedua lock
Thread2: mendapat kedua lock
madha_27@DESKTOP-S58N8I3:~$ |

```

2.3. Analisis Hasil

dua teknik pencegahan deadlock diuji: Lock Ordering dan Try-Lock. Pada metode Lock Ordering, kedua thread selalu mengambil lock dengan urutan yang sama. Hasilnya, program berjalan lancar tanpa macet karena tidak ada kondisi saling tunggu. Metode ini paling efektif dan tidak membutuhkan percobaan ulang.

Pada metode Try-Lock, thread mencoba mengambil lock kedua. Jika gagal, lock dilepas dan thread mencoba lagi. Hasilnya tetap tidak terjadi deadlock, namun thread beberapa kali harus mengulang.

Teknik ini lebih fleksibel tetapi kurang efisien karena membutuhkan retry.

2.4. Perbandingan lock ordering vs try lock

Metode	Kelebihan	Kekurangan
Lock_ordering	Paling aman, sederhana, dijamin tidak deadlock.	Harus semua thread mengikuti urutan lock yang sama.
Try_lock	Fleksibel, thread tidak harus berurutan	Lebih boros CPU karena retry terus

3. TUGAS 3 BANKER'S ALGORITHM

3.1. Kode Pemrograman

```
#include <stdio.h>
#include <stdbool.h>

#define P 4
#define R 3

int available[R] = {5,4,3};

int maximum[P][R] = {
    {7,5,3},
    {3,2,2},
    {9,0,2},
    {2,2,2}
};

int allocation[P][R] = {
    {0,1,0},
    {2,0,0},
    {3,0,2},
    {2,1,1}
};

int need[P][R];

void calc_need() {
    for(int i = 0; i < P; i++)
        for(int j = 0; j < R; j++)
            need[i][j] = maximum[i][j] - allocation[i][j];
}

bool is_safe() {
    int work[R];
    bool finish[P] = {0};
    int seq[P], idx = 0;

    for(int i=0;i<R;i++) work[i] = available[i];

    while(idx < P) {
        bool found = false;
```

```

        for(int i=0;i<P;i++) {
            if(!finish[i]) {
                bool ok = true;

                for(int j=0;j<R;j++)
                    if(need[i][j] > work[j]) ok = false;

                if(ok) {
                    for(int j=0;j<R;j++)
                        work[j] += allocation[i][j];
                    seq[idx++] = i;
                    finish[i] = true;
                    found = true;
                }
            }

            if(!found) return false;
        }

        printf("Safe sequence: ");
        for(int i=0;i<P;i++) printf("P%d ", seq[i]);
        printf("\n");

        return true;
    }

bool request(int p, int req[]) {
    // cek need
    for(int i=0;i<R;i++)
        if(req[i] > need[p][i]) return false;

    // cek available
    for(int i=0;i<R;i++)
        if(req[i] > available[i]) return false;

    // simulasi alokasi
    for(int i=0;i<R;i++) {
        available[i] -= req[i];

```

```

        // cek need
        for(int i=0;i<R;i++)
            if(req[i] > need[p][i]) return false;

        // cek available
        for(int i=0;i<R;i++)
            if(req[i] > available[i]) return false;

        // simulasi alokasi
        for(int i=0;i<R;i++) {
            available[i] -= req[i];
            allocation[p][i] += req[i];
            need[p][i] -= req[i];
        }

        if(is_safe()) return true;

        // rollback
        for(int i=0;i<R;i++) {
            available[i] += req[i];
            allocation[p][i] -= req[i];
            need[p][i] += req[i];
        }

        return false;
    }

int main() {
    calc_need();
    is_safe();

    int r1[3] = {1,0,2};
    request(1, r1);

    int r2[3] = {3,3,0};
    request(3, r2);

    return 0;
}

```

3.2. Testing Output

```
madha_27@DESKTOP-S58N8I3:~$ nano banker.c
madha_27@DESKTOP-S58N8I3:~$ gcc banker.c -o banker -pthread
madha_27@DESKTOP-S58N8I3:~$ ./banker
Safe sequence: P1 P2 P3 P0
Safe sequence: P1 P2 P3 P0
madha_27@DESKTOP-S58N8I3:~$ |
```

3.3. Analisis Hasil

Hasil program banker menunjukkan safe sequence p1 p2 p3 p0 ini berarti system berada dalam kondisi aman, karena masih ada urutan proses yang dapat dijalankan tanpa menyebabkan deadlock. Setiap proses dapat berjalan setelah proses sebelumnya selesai dan mengembalikan resource. Dengan demikian, alokasi resource yang digunakan tidak berisiko menimbulkan deadlock, dan semua proses bisa diselesaikan secara berurutan.

4. TUGAS 4 KASUS NYATA: DEADLOCK TRANSFER UANG

4.1. Kode Pemrograman

```
#include <stdio.h>
#include <pthread.h>
#include <unistd.h>

pthread_mutex_t rek1 = PTHREAD_MUTEX_INITIALIZER;
pthread_mutex_t rek2 = PTHREAD_MUTEX_INITIALIZER;

// Selaluurut: rek1 → rek2
void lock_account() {
    pthread_mutex_lock(&rek1);
    pthread_mutex_lock(&rek2);
}

void unlock_account() {
    pthread_mutex_unlock(&rek2);
    pthread_mutex_unlock(&rek1);
}

void* transferA(void* arg) {
    lock_account();
    printf("A: Transfer dari Rek1 ke Rek2...\n");
    sleep(1);
    unlock_account();
    printf("A selesai\n");
    return NULL;
}

void* transferB(void* arg) {
    lock_account();
    printf("B: Transfer dari Rek2 ke Rek1...\n");
    sleep(1);
    unlock_account();
    printf("B selesai\n");
    return NULL;
}

int main() {
    pthread_t t1, t2;
    pthread_create(&t1, NULL, transferA, NULL);
    pthread_create(&t2, NULL, transferB, NULL);
```



```
pthread_join(t1, NULL);
pthread_join(t2, NULL);

return 0;
}
```

4.2. Testing Output

```
madha_27@DESKTOP-S58N8I3:~$ nano lock_ordering_rek.c
madha_27@DESKTOP-S58N8I3:~$ gcc lock_ordering_rek.c -o lock_ordering_rek -pthread
madha_27@DESKTOP-S58N8I3:~$ ./lock_ordering_rek
A: Transfer dari Rek1 ke Rek2...
A selesai
B: Transfer dari Rek2 ke Rek1...
B selesai
madha_27@DESKTOP-S58N8I3:~$ |
```

4.3. Analisis Hasil

dua proses transfer uang saling mengakses rekening secara bersamaan. Jika keduanya mengunci rekening secara acak, deadlock bisa terjadi karena saling menunggu. Setelah program dijalankan dengan teknik lock ordering, hasilnya menunjukkan bahwa kedua proses dapat selesai tanpa macet. Hal ini terjadi karena kedua thread mengambil kunci rekening dengan urutan yang sama, sehingga tidak ada kondisi saling tunggu. Dengan urutan penguncian yang konsisten, deadlock berhasil dicegah dan proses transfer berjalan lancar.

5. KESIMPULAN

Deadlock adalah kondisi ketika dua proses saling menunggu resource sehingga tidak ada yang bisa lanjut. Pada percobaan awal, deadlock terjadi karena kedua thread mengambil lock secara saling silang, sehingga memenuhi empat syarat deadlock: mutual exclusion, hold and wait, no preemption, dan circular wait.

Deadlock dapat dideteksi menggunakan timeout, yaitu ketika thread terlalu lama menunggu lock. Namun deteksi hanya memberi tahu bahwa deadlock sudah terjadi, bukan mencegahnya.

Untuk mencegah deadlock, digunakan dua teknik utama. Pertama, lock ordering, yaitu semua thread mengambil lock dengan urutan yang sama sehingga circular wait hilang dan deadlock tidak mungkin terjadi. Kedua, try-lock, yaitu thread mencoba mengambil lock, dan jika gagal, lock dilepas dan dicoba lagi. Teknik ini fleksibel tetapi lebih sering mengulang.

Banker's Algorithm digunakan untuk memastikan resource hanya diberikan jika sistem masih berada dalam kondisi aman. Jika pemberian resource menyebabkan kondisi tidak aman, permintaan ditolak agar tidak terjadi deadlock.

Kasus nyata seperti transfer uang antar rekening juga bisa mengalami deadlock jika kedua proses mengunci rekening secara acak. Dengan menerapkan lock ordering, sistem dapat berjalan aman tanpa macet.

Secara singkat, deadlock bisa terjadi kapan saja saat resource dipakai bersama, tetapi dapat dicegah dengan aturan yang benar, urutan pengambilan lock yang konsisten, atau pengecekan keamanan sistem seperti Banker's Algorithm.